

Uso de Fixed-Point em ECUs para Reduzir Dependencia de float

Projeto: VMU - Vehicle Management Unit

Artefato: Relatorio tecnico

Data: 2026-06-08

Escopo: boas praticas gerais para ECUs/embedded C e aplicacao ao projeto VMU.

Sumario

Uso de Fixed-Point em ECUs para Reduzir Dependencia de `float`	1
Sumario Executivo	2
Por que ECUs Restringem `float`	2
FPU pode ser ausente, opcional ou limitada	2
Economia de memoria nao e apenas o tamanho da variavel	2
Previsibilidade temporal e WCET	3
Portabilidade e reprodutibilidade numerica	3
Tratamento Recomendado: Inteiros Escalados e Fixed-Point	3
Inteiros escalados	3
Q-format	3
Saturacao, overflow e arredondamento	3
Relacao com MISRA C e AUTOSAR	4
Fluxo Model-Based: Simulink em Unidades Fisicas, C em Escala Inteira	4
Estudo de Caso: VMU	5
Checklist para Novos Modulos de ECU	5
Recomendacoes para a VMU	6
Referencias	6

Sumario Executivo

Em ECUs automotivas e sistemas embarcados de tempo real, o uso de `float` nao deve ser tratado apenas como uma escolha de sintaxe. Ele afeta previsibilidade temporal, portabilidade numerica, custo de verificacao, dependencia de FPU, bibliotecas de suporte e, em muitos casos, consumo de Flash/RAM. A recomendacao pratica nao e afirmar que `float` e sempre proibido, mas restringir seu uso em software de producao quando a funcao pode ser representada com inteiros escalados ou fixed-point com faixa e precisao conhecidas.

O padrao mais usado em software embarcado critico e:

- 1 Manter grandezas fisicas em unidades claras no modelo e na documentacao.
- 2 Definir uma representacao inteira para a interface C, como decimos de km/h, decimos de kW ou porcentagem em base 10000.
- 3 Documentar escala, faixa, resolucao, saturacao e arredondamento.
- 4 Evitar literais `float` e conversoes implicitas em headers e logica de producao.
- 5 Validar equivalencia numerica contra o modelo de referencia e testar bordas, overflow e histerese.

No projeto VMU, a API atual ja segue essa direcao: `speed_dkph`, `p_dem_dkw`, `soc_q10000` e `weng_rpm` usam tipos inteiros com escala explicita, e os thresholds de decisao tambem estao em macros inteiras. Isso reduz a dependencia de ponto flutuante na logica de modo e deixa o comportamento mais adequado a MISRA C e a execucao em ECUs sem FPU dedicada.

Por que ECUs Restringem `float`

FPU pode ser ausente, opcional ou limitada

Nem todo microcontrolador usado em ECU possui FPU. Mesmo em familias automotivas modernas, a FPU pode ser opcional ou variar entre variantes do hardware. A documentacao da Infineon para AURIX TriCore descreve FPU como recurso opcional em sua arquitetura, enquanto uma nota de aplicacao para controlador baseado em ARM Cortex-M0 declara explicitamente que aquele nucleo nao possui FPU e suporta aritmetica fixed-point. Isso significa que um mesmo codigo C com `float` pode ter custo muito diferente dependendo do alvo.

Quando nao ha FPU, operacoes de ponto flutuante sao implementadas em software por rotinas auxiliares. O impacto tipico e:

- aumento de Flash por chamadas a bibliotecas de suporte;
- maior latencia por operacao;
- maior variacao de tempo em multiplicacoes, divisoes e funcoes matematicas;
- maior dificuldade para estimar WCET;
- risco de dependencias indesejadas em `libm` ou runtime do compilador.

Economia de memoria nao e apenas o tamanho da variavel

Um `float` de 32 bits pode ter o mesmo tamanho bruto de um `uint32_t`, mas essa comparacao isolada e incompleta. Em codigo embarcado, o ganho de memoria vem de tres pontos:

- usar tipos menores quando a faixa permite, como `uint8_t`, `uint16_t` ou `int16_t`;
- evitar bibliotecas de ponto flutuante e funcoes matematicas auxiliares;
- reduzir alinhamentos, temporarios intermediarios e buffers maiores que o necessario.

Exemplo: uma velocidade com resolucao de 0,1 km/h ate 6553,5 km/h cabe em `uint16_t`. Para uma ECU veicular, essa faixa e muito maior que o necessario. Nesse caso, `float` nao agrega capacidade funcional para a logica de estados, mas aumenta a superficie de conversao e verificacao.

Previsibilidade temporal e WCET

Software de ECU normalmente executa em tarefas periodicas, com orçamento de tempo fixo. A logica precisa terminar dentro do periodo da tarefa em todos os casos relevantes, nao apenas no caso medio. Fixed-point e inteiros escalados tornam o custo de comparacoes, somas e subtracoes mais previsivel. Tambem facilitam revisao manual e analise estatica, porque a faixa numerica pode ser expressa diretamente no tipo e nas macros de calibracao.

Para controle supervisorio, como selecao de modo da VMU, a maior parte da logica e comparacao de thresholds. Nesse tipo de decisao, `float` geralmente nao e necessario se a resolucao escolhida cobre os requisitos fisicos.

Portabilidade e reprodutibilidade numerica

Ponto flutuante moderno pode ser deterministico dentro de um alvo bem definido, compilador controlado e configuracao de FPU conhecida. O problema pratico em projetos automotivos e que a cadeia de ferramentas pode envolver simulacao em host, SIL/PIL, diferentes opcoes de compilador, diferentes MCUs e geracao automatica de codigo. Pequenas diferencas de arredondamento, promocao para `double`, tratamento de denormais ou otimizacoes podem dificultar comparacoes bit-a-bit.

Fixed-point tambem exige cuidado, especialmente com overflow e arredondamento. A vantagem e que essas regras podem ser definidas explicitamente: escala, saturacao, largura do acumulador e modo de arredondamento passam a fazer parte da especificacao.

Tratamento Recomendado: Inteiros Escalados e Fixed-Point

Inteiros escalados

Inteiro escalado e a forma mais simples de fixed-point. Uma grandeza fisica e armazenada como inteiro multiplicado por uma escala fixa:

Grandeza fisica	Representacao	Exemplo
Velocidade em km/h	<code>speed_dkph = km/h * 10</code>	35,0 km/h -> 350
Potencia em kW	<code>p_dem_dkw = kW * 10</code>	-5,0 kW -> -50
SOC em %	<code>soc_q10000 = fracao * 10000</code>	37% -> 3700
Rotacao em rpm	<code>weng_rpm = rpm</code>	800 rpm -> 800

Esse padrao e apropriado para thresholds, histerese, maquinas de estado, diagnosticos simples e interfaces de controle onde a resolucao necessaria e conhecida.

Q-format

Q-format e uma representacao fixed-point em que parte dos bits representa a fracao. Formatos como Q15, Q31 ou Q7 aparecem em bibliotecas de DSP, incluindo CMSIS-DSP da Arm. O ponto importante e que multiplicacao e acumulacao precisam de regras claras:

- multiplicar dois valores fixed-point aumenta a largura intermediaria;
- o resultado precisa ser deslocado para voltar a escala original;
- overflow deve ser evitado por projeto ou tratado com saturacao;
- comparacoes so sao seguras quando as escalas sao compativeis.

Para a VMU, a logica atual nao precisa de Q-format complexo, porque as decisoes sao majoritariamente comparacoes contra thresholds. Inteiros escalados sao mais simples e suficientes.

Saturacao, overflow e arredondamento

O erro mais comum ao substituir `float` por inteiro e ignorar overflow intermediario. A regra de projeto deve ser:

- definir faixa fisica maxima e minima de cada entrada;
- escolher tipo com margem;
- usar acumulador mais largo em multiplicacao ou soma acumulada;
- aplicar saturacao quando a falha por wraparound for inaceitavel;
- documentar arredondamento em conversoes de unidade fisica para escala inteira.

Para logica de modo, thresholds devem ser expressos ja na unidade escalada. Isso evita conversoes em runtime e reduz risco de erro:

```
#define SPEED_STOP_DKPH    (5U)      /* 0.5 km/h */
#define PDEM_REGEN_DKW     (-50)     /* -5.0 kW */
#define SOC_EV_IN_Q10000   (3700U)  /* 37.00% */
```

Relacao com MISRA C e AUTOSAR

MISRA C nao deve ser interpretado como uma proibicao absoluta de `float`. O foco das regras e reduzir comportamento indefinido, conversoes perigosas, perda de precisao, dependencia de implementacao e ambiguidades de tipo. A familia de regras MISRA C:2012 Rule 10.x usa o conceito de "essential type model" para controlar atribuicoes, conversoes e operacoes entre categorias de tipos.

Pontos praticos para um codigo mais alinhado a MISRA C:

- evitar conversoes implicitas entre `float` e inteiros;
- evitar atribuir uma expressao a um tipo mais estreito sem justificativa;
- evitar misturar signed e unsigned na mesma expressao;
- usar tipos de largura fixa de `<stdint.h>`;
- deixar thresholds e escalas em macros ou constantes tipadas;
- evitar literais `float` em headers de producao quando a API e inteira;
- validar estaticamente com ferramentas como cppcheck/MISRA, Polyspace ou equivalentes.

AUTOSAR C++14 tambem restringe conversoes implicitas entre tipos de ponto flutuante e tipos inteiros, o que reforca a mesma direcao de projeto: se a conversao for necessaria, ela deve ser explicita, localizada, revisada e testada.

Fluxo Model-Based: Simulink em Unidades Fisicas, C em Escala Inteira

Em desenvolvimento baseado em modelos, e comum manter o Simulink/Stateflow em unidades fisicas porque isso facilita validacao por engenheiros de sistema, calibracao e revisao de requisitos. A conversao para C embarcado deve ocorrer em uma fronteira bem definida:

- 1 O modelo usa km/h, kW, SOC e rpm em unidades de engenharia.
- 2 O wrapper de teste converte essas grandezas para a escala C.
- 3 A API C recebe apenas inteiros escalados.
- 4 A comparacao de equivalencia converte as saidas para o mesmo dominio sem alterar a semantica.

Ferramentas como Fixed-Point Designer e Embedded Coder suportam geracao de codigo fixed-point a partir de modelos, incluindo codigo que usa tipos inteiros e deslocamentos para representar posicoes de ponto fixo. A documentacao da MathWorks tambem destaca teste numerico, deteccao de overflow, SIL/PIL, rastreabilidade e cobertura como partes do fluxo de verificacao.

Para a VMU, a abordagem recomendada e manter o Stateflow como referencia fisica e preservar a API C fixed-point. A equivalencia deve ser testada em bordas de thresholds e histerese, como `SOC_EV_IN`, `SOC_EV_OUT`, `ENG_ON`, `ENG_OFF`,

Estudo de Caso: VMU

O header `inc/mode_logic_team.h` já define uma interface embarcada com escalas inteiras:

Campo	Tipo	Escala	Uso
<code>speed_dkph</code>	<code>uint16_t</code>	0,1 km/h	velocidade do veiculo
<code>p_dem_dkw</code>	<code>int16_t</code>	0,1 kW	potencia demandada, incluindo regeneracao negativa
<code>soc_q10000</code>	<code>uint16_t</code>	0..10000	estado de carga da bateria
<code>weng_rpm</code>	<code>uint16_t</code>	rpm inteiro	velocidade do motor a combustao

Os thresholds de controle tambem estao expressos como inteiros:

Macro	Valor	Significado fisico
<code>SPEED_STOP_DKPH</code>	5U	0,5 km/h
<code>SPEED_REGEN_DKPH</code>	50U	5,0 km/h
<code>SPEED_EV_MAX_DKPH</code>	350U	35,0 km/h
<code>PDEM_REGEN_DKW</code>	-50	-5,0 kW
<code>PDEM_HYB_IN_DKW</code>	500	50,0 kW
<code>PDEM_HYB_OUT_DKW</code>	400	40,0 kW
<code>SOC_EV_IN_Q10000</code>	3700U	37,00%
<code>SOC_EV_OUT_Q10000</code>	3500U	35,00%
<code>ENG_ON_RPM</code>	800U	800 rpm
<code>ENG_OFF_RPM</code>	790U	790 rpm

Essa decisao e adequada para a natureza da logica da VMU:

- as transicoes sao baseadas em comparacoes e histerese;
- a resolucao de 0,1 km/h e 0,1 kW e suficiente para thresholds supervisórios;
- SOC em base 10000 permite expressar porcentagens com duas casas decimais;
- rpm inteiro e natural para o sinal de velocidade do motor;
- `Outputs_t` usa `uint8_t` para saidas binarias, reduzindo tamanho em comparacao com `int`.

O cuidado principal e manter a conversao fisica -> inteiro fora da logica central, com saturacao ou validacao de faixa antes de chamar `ModeLogic_Step`. Assim, a maquina de estados nao precisa lidar com `float`, `double` ou conversoes implicitas.

Checklist para Novos Modulos de ECU

- 1 Definir a unidade fisica de cada sinal.
- 2 Definir resolucao minima exigida pelo requisito.
- 3 Escolher escala inteira que cubra faixa e resolucao com margem.
- 4 Escolher tipo de largura fixa (`uint8_t`, `int16_t`, `uint16_t`, `int32_t`).
- 5 Documentar valor fisico, escala, minimo, maximo e comportamento fora de faixa.
- 6 Definir saturacao, truncamento ou arredondamento de cada conversao.
- 7 Evitar literais `float` e `double` em headers de producao.

- 8 Evitar conversoes implicitas entre inteiros, signed/unsigned e floating-point.
- 9 Usar acumuladores mais largos em multiplicacao e soma acumulada.
- 10 Testar bordas: minimo, maximo, threshold - 1 LSB, threshold e threshold + 1 LSB.
- 11 Validar equivalencia com o modelo em unidades fisicas.
- 12 Rodar analise estatica MISRA e revisar todos os desvios.

Recomendacoes para a VMU

- Manter a API publica em fixed-point inteiro.
- Manter thresholds em macros inteiras com comentario de valor fisico.
- Concentrar conversoes fisicas em wrappers de teste, adaptadores de integracao ou camada de aquisicao de sinais.
- Nao reintroduzir float em inc/mode_logic_team.h nem em guards de src/mode_logic_team.c.
- Quando o modelo Simulink usar unidades fisicas, documentar a conversao aplicada nos testes de equivalencia.
- Para qualquer nova guarda, escrever teste nos tres pontos relevantes: abaixo do threshold, no threshold e acima do threshold.

Referencias

- 1 MathWorks, [Fixed-Point Code Generation in Simulink](#).
- 2 MathWorks, [Code Generation by Using Embedded Coder](#).
- 3 MathWorks, [Automated Fixed-Point Conversion](#).
- 4 MathWorks, [Embedded Code Generation - Production Code in Automotive ECUs](#).
- 5 Arm, [CMSIS-DSP Fixed Point Datatypes](#).
- 6 MathWorks/Polyspace, [MISRA C:2012 Rule 10.3](#).
- 7 MathWorks/Polyspace, [Essential Type Model Used in MISRA C Rule Checking](#).
- 8 MathWorks/Polyspace, [AUTOSAR C++14 Rule M5-0-5](#).
- 9 Infineon, [32-bit AURIX TriCore Microcontroller](#).
- 10 Infineon, [XDPP1100 Getting Started Firmware Development Application Note](#).